

Cartiae Rae — Full Audit

Date: 2026-08-09 **Scope:** Whole application — payments, auth, data architecture, deployment state, content integrity. **Codebase:** 38 TS/TSX files, ~12,645 LOC. Live at `cartiaerae.netlify.app`.

Supersedes: `audit_report.md` (the earlier Tier-1 security pass).

Headline: The storefront is one change away from a serious payment flaw: **item prices are supplied by the browser and passed to Stripe unmodified**, on a **live** Stripe key. Separately, once configuration is completed the admin Orders Ledger will stay permanently empty, because nothing in the frontend ever reads the `orders` table that the webhook writes to. Both are fixable; neither is subtle once you look for it.

Severity legend

	Meaning
CRITICAL	Exploitable now, causes direct financial or data loss
HIGH	Core promised functionality does not work, or a real security gap
MEDIUM	Correctness/maintainability risk; degrades over time
LOW	Hygiene, performance, polish

CRITICAL

C1 — The browser sets the price it pays — FIXED

Resolved 2026-08-09, verified against the live endpoint. Prices and discount percentages are now resolved server-side from `site_snapshots`; the client sends only ids and quantities. Verified by posting a tampered cart (`price: 0.50` , `name: "HACKED"` , plus a fabricated `discountPercent: 99`) to the production function and rendering the resulting Stripe page: it shows **\$24.99** and **"The 4C Growth Blueprint"**. An unknown item id is rejected with *"One of the items in your cart is no longer available."* The function fails **closed** — if the catalog cannot be read it returns 503 rather than falling back to client prices.

The original finding is preserved below for the record.

`netlify/functions/create-checkout-session.js:130-147`

```
const lineItems = cart.map((item) => {
  const unitAmountCents = Math.round(item.price * discountFactor * 100);
  ...
  unit_amount: Math.max(unitAmountCents, 50),
```

`item.price` comes straight from the POST body. `validateCartItems` only checks that it is a number between 0 and 10000 — it never compares against a catalog. Anyone can open DevTools, edit the request, and buy a \$200 product for \$0.50.

`appliedDiscount.discountPercent` is equally client-controlled and isn't validated at all — not even by `validateCartItems`. Sending `discountPercent: 100` drives every line to Stripe's \$0.50 floor.

This is live, not theoretical: the deployed bundle contains a `pk_live_` key, so the store is taking real money.

Fix: the server must own prices. Look each `item.id` up in a Supabase catalog table (or `ebook_files` - style mapping) and build `unit_amount` from the stored value, ignoring whatever the client sent. Same for discount codes — resolve the code server-side and read its percentage from the database.

Until that lands, the exposure is bounded by how many people know the endpoint exists.

HIGH

H1 — Real orders will never appear in the admin — FIXED (needs SQL)

Resolved in code 2026-08-09. `AppContext` now fetches `public.orders` when an admin is logged in, and "Mark Dispatched" persists via an `update` instead of only mutating local state. Two supporting pieces ship in `supabase/orders_ebooks_setup.sql` and must be run: an `orders admin read` policy (without it the ledger authenticates and renders zero rows) and an `orders admin update` policy.

The Stripe webhook writes verified orders to `public.orders`. But **no frontend code ever reads that table** — `grep "from('orders')" src` returns nothing. The Orders Ledger renders the React `orders` state, which is seeded from `localStorage`.

Consequence: after configuration is finished, a real customer pays, the webhook records the order correctly, and the studio owner sees an empty ledger forever.

This was previously *masked* by two hardcoded demo orders (Aria Carter, Shayla Jenkins). Those are now correctly gated off when connected, which makes the gap visible rather than causing it.

Fix: mirror the pattern already used for `contactRequests` at `AppContext.tsx:469` — fetch `orders` from Supabase when an admin is logged in. Roughly 20 lines.

H2 — Three tables are used but never created — migration written; partly disproved

Correction, 2026-08-09. `site_snapshots` **does exist and has data** — proven when the new server-side pricing successfully loaded the catalog from it in production. My original claim that it was missing was wrong; only the *migration* was missing from the repo. `videos` and `gallery_items` remain unverified. `supabase/site_content_setup.sql` now creates all three idempotently with RLS (public read, admin write) and seeds the catalog, so it is safe to run either way.

Table	Used at	Defined in <code>supabase/*.sql</code> ?
<code>site_snapshots</code>	<code>AppContext.tsx:526</code> , <code>:1219</code>	No
<code>videos</code>	<code>AppContext.tsx:790</code> , <code>841</code> , <code>855</code>	No
<code>gallery_items</code>	<code>AppContext.tsx:873</code> , <code>895</code> , <code>909</code>	No

`site_snapshots` is the one that matters most: it is how the owner's edits reach visitors and other devices. If the table is absent, she edits, sees her changes locally, gets a "Sync failed" toast, and the public site never updates.

Fix: write the migration for all three. I can generate it.

H3 — Production is missing one required secret

Probing the live functions returns, verbatim:

```
stripe-webhook      → 500 {"missing":["STRIPE_WEBHOOK_SECRET"]}
get-ebook-download  → 400 "A valid Stripe session id is required." (configured)
create-checkout-session → 400 "A valid customer email is required." (configured)
```

The Supabase key was present all along under the name `SUPABASE_SECRET_KEY`, while the functions read `SUPABASE_SERVICE_ROLE_KEY` — a silent name mismatch. The functions now accept either name, which resolved `get-ebook-download`. Only the Stripe webhook secret remains, and it cannot exist until the endpoint is registered in Stripe.

Checkout works; **recording the sale does not**. On a live key that means money can be taken with no order row, no eBook delivery, and no record for the studio. The webhook returns 500 so Stripe will retry for ~3 days — orders are recoverable if the secrets are added inside that window, and lost after it.

Fix: set `STRIPE_WEBHOOK_SECRET` and `SUPABASE_SERVICE_ROLE_KEY` in Netlify with **Scopes: All** (Builds-only is the usual reason a correctly-typed variable is invisible to functions), and register the endpoint in Stripe for `checkout.session.completed`.

H4 — `orders_ebooks_setup.sql` has not been run

The `orders`, `ebook_files` tables and the private `ebooks` bucket do not exist yet. Even with the secrets set, the webhook would verify the payment and then fail to store it.

MEDIUM

M1 — Roles are stored but never enforced

`admin_users.role` accepts `super_admin` / `store_manager` / `content_manager`, and `auth_full_setup.sql` promotes everyone to `super_admin`. But a search for `role ===`, `hasPermission`, or `canManage` across `src/` returns **zero matches**.

Login checks only that a row exists (`AppContext.tsx:1136–1145`). Every admin can do everything. The three-tier role system is presentational.

Fine for a single-owner studio; a real problem the moment an assistant is added.

M2 — `localStorage` is the source of truth for the catalog

Products, eBooks, videos, gallery, blogs and services all initialise from `localStorage` with `initialData.ts` as fallback. Supabase is a *snapshot mirror*, not the primary store. Implications:

- Two admins on two machines diverge silently; last publish wins.
- Clearing browser data loses unpublished work.
- `localStorage` caps near ~5 MB. The code has quota-handling at `AppContext.tsx:574`, which tells you this has already been hit — base64 images overflow it quickly.

M3 — XSS via admin-controlled content — FIXED

Resolved 2026-08-09. The fallback label is now built with `createElement` + `textContent` instead of interpolated `innerHTML`.

`src/views/ServicesPage.tsx:75`

```
fb.innerHTML = `${service.name}</span>`;
```

`service.name` is interpolated into raw HTML on image-load failure. Admin-only input, so severity is limited — but it is a stored-XSS path that executes for every visitor. Use `textContent`.

M4 — The repository is public

`themainkeys/Cartie-Rae` is **public** (`"private": false`). Anything committed is world-readable and indexed. Currently clean — I found no real key values in tracked files, `.env` is gitignored and untracked, and only variable *names* appear in code and docs. Keep it that way: a `service_role` key committed here bypasses every RLS policy in the database.

M5 — An expired PAT was embedded in the git remote

`.git/config` carried `https://ghp_...@github.com/...`. It is dead (auth fails), but it was stored in plaintext and printed by `git remote -v`. Revoke rather than replace, and prefer `gh auth login` or a credential helper over a token in the URL.

LOW

L1 — Single 748 KB JS bundle

`dist/assets/index-*.js` is ~748 KB (209 KB gzipped) and exceeds Vite's warning threshold. `AdminPortal` is already lazy-loaded, which is the right instinct; the storefront chunk itself is the remaining weight. Not urgent.

L2 — Demo session is a tamper-resistant-ish localStorage flag

`readValidDemoSession` validates shape and expiry only. This is correct and honestly labeled — demo mode is hard-disabled whenever Supabase is configured (`demoLogin` returns early at `AppContext.tsx:1159`). No action needed.

Fixed since the previous audit

Item	Status
Hardcoded plaintext passwords (<code>admin</code> , <code>manager</code> , ...) in <code>api.ts</code>	Removed (commit <code>d219363</code>)
Real Supabase Auth login + <code>admin_users</code> role lookup	Implemented
Stripe webhook with signature verification and idempotent upsert	Added (<code>stripe-webhook.js</code>)
Secure signed-URL eBook delivery, private bucket, no client-supplied paths	Added (<code>get-ebook-download.js</code>)
Two invented customer orders in the ledger	Gated off when connected
Fabricated like counts + comments on all 17 videos	Gated off when connected
Invented product/eBook testimonials	Stripped when connected

Discovered while fixing: the Functions runtime has no WebSocket

Adding `@supabase/supabase-js` to a Netlify Function crashes it at `createClient()` :

Node.js 20 detected without native WebSocket support

The full client always constructs a Realtime client, which needs a WebSocket that Node 20 lacks natively. **This briefly took live checkout down** and is worth recording, because the obvious fix does not work: the Functions runtime version is set by `AWS_LAMBDA_JS_RUNTIME`, **not** by `NODE_VERSION` in `netlify.toml`, so bumping the build image changes nothing.

All three functions now talk to PostgREST and Storage directly via `fetch` (`netlify/functions/lib/supabaseRest.js`). No WebSocket, no realtime, no dependency — and a smaller function bundle. Anyone reintroducing `supabase-js` into a function will hit this again.

Remaining work — what is still blocked

#	Item	Blocked on
1	<code>STRIPE_WEBHOOK_SECRET</code>	Registering the endpoint in Stripe; the value does not exist until then
2	Run <code>supabase/orders_ebooks_setup.sql</code>	Dashboard access. <code>orders</code> does not exist yet — confirmed by <code>get-ebook-download</code> failing its lookup in production
3	Run <code>supabase/site_content_setup.sql</code>	Dashboard access
4	Upload the three eBook PDFs to the private <code>ebooks</code> bucket	Dashboard access + the files
5	M1 (roles), M2 (localStorage architecture)	Deliberate scheduling, not urgent

Everything else in this report is done and deployed.

Recommended order of work

1. **C1** — server-side pricing. Live key, real money, exploitable today.
2. **H3 + H4** — secrets and SQL, so paid orders stop being dropped.
3. **H1** — surface real orders in the admin, or the studio is blind to its own sales.
4. **H2** — `site_snapshots` migration, or content publishing silently fails.
5. **M3** — one-line `textContent` fix.
6. **M1, M2** — architectural; schedule deliberately, not in a rush.

Items 1–4 are what stand between this and a store that can safely take money and be handed to its owner.